

Omar Joaquín Sosa Moreno 1997991 IAS

Tema 1 Modelo OSI Protocolo TCP/IP 13 12 26

Modelo OSI (Open System Interconnection)

El modelo OSI es un marco de referencia conceptual creado por la ISO que describe cómo deben comunicarse los sistemas en una red. Se divide en 7 capas, donde cada una tiene una función específica.

7 capas del modelo OSI

1. Capa Física: Se encarga de la transmisión de bits puros a través de un medio físico (cables, ondas de radio).

2. Capa de Enlace de datos: Organiza los bits en "Tramas" y gestiona la detección de errores en el medio físico.

3. Capa de Red: Determina la mejor ruta para enviar paquetes de datos de un punto a otro (Direccionamiento lógico/IP)

4. Capa de Transporte: Garantiza que los mensajes se entreguen sin errores y en secuencia (Control de flujo)

5. Capa de Sesión: Administra y mantiene las conexiones (sesiones) entre aplicaciones

6. Capa de presentación: Traduce el formato de los datos (cifrado, compresión) para que la capa de aplicación los entienda.

7. Capa de Aplicación: Es la interfaz con la que interactúa el usuario final (navegadores, correo)

Modelo TCP/IP

Es el marco conceptual estándar utilizado para la interconexión de redes, compuesto por cuatro o cinco capas fundamentales. Define cómo se empaquetan, direccionan, transmiten y reciben los datos en internet, sustituyendo al modelo OSI por ser más simple.

1. Capa de aplicación: Donde residen las aplicaciones de red y sus protocolos. Es el nivel de software con el que interactuamos.

2. Capa de Transporte: Se encarga de la transferencia de datos entre procesos (host to host). Aquí es donde se decide si la conexión es segura y ordenada (TCP) o rápida (UDP).

3. Capa de Red / Internet: Su misión es mover los paquetes desde el origen al destino a través de diferentes redes. Aquí "viven" las direcciones IP.

4. Capa de Enlace de Datos: Se encarga del direccionamiento físico (direcciones MAC) y de cómo se empaquetan los datos para viajar por cable o el aire.

5. Capa Física: Define las características del medio físico (Voltajes, tipos de cables, conectores).

Protocolos de aplicación (Nivel de ~~Aplicación~~)

1. HTTP: Protocolo de transferencia de hipertexto sirve para cargar páginas web.
2. HTTPS: Versión segura del HTTP que utiliza cifrado (SSL/TLS).
3. FTP: Protocolo para transferir archivos entre un cliente y un servidor.
4. SMTP: Se encarga del envío de correos electrónicos.
5. POP3: Permite descargar correos electrónicos desde un servidor al cliente local.
6. IMAP: Permite visualizar y gestionar correos directamente en el servidor.
7. DNS: Traduce nombres de dominio en direcciones IP.
8. DHCP: Asigna direcciones IP de forma automática a los dispositivos de una red.
9. SSH: Permite el acceso remoto seguro a otros dispositivos.
10. Telnet: Similar a SSH pero inseguro, usado para gestión remota antigua.
11. SNMP: Utilizado para gestionar y monitorear dispositivos de red (routers, switches).
12. TCP: Orientado a conexiones; garantiza que los datos lleguen completos y en orden.
13. UDP: No orientado a conexión; es más rápido pero no garantiza la llegada de datos.
14. IP (IPv4/IPv6): Se encarga del direccionamiento y envío de paquetes a través de la red.
15. ICMP: Utilizado para enviar mensajes de error y diagnóstico.
16. ARP: Traduce una dirección IP a una dirección física.

Omar Joaquín Sosa Moreno 1997910 TAS

13 12 26

17. BGP: Protocolo que decide las rutas de tráfico en el internet global.
18. OSPF: Protocolo de enrutamiento interno para encontrar el camino mas corto en una red privada.
19. Ethernet: Define cómo se envían los datos a través de cables físicos en una red local.
20. TLS/SSL: Protocolos de seguridad que cifran la comunicación en la red.

10 Sucesos Importantes en la historia del internet

1. ARPANET (1969): Se realiza la primera conexión entre computadoras de las universidades de UCLA y Stanford. Es considerado el abuelo del internet.

2. Protocolo TCP/IP (1983): Se adopta como el estándar de comunicación oficial, permitiendo que diferentes redes se entendieran entre sí.

3. Nacimiento de la World Wide Web (1989-1990): El primer navegador aún no existía, Tim Berners-Lee inventa el HTML, el HTTP y el primer navegador en el CERN.

4. Lanzamiento de Mosaic (1993): El primer navegador web que permitió visualizar imágenes junto al texto, haciendo la red accesible para personas.

5. Comercialización y buscadores (1995-1998): Nacen gigantes como Amazon, eBay y finalmente, Google, transformando la forma en que compramos e investigamos.

6. La burbuja de las "Dot-com" (2000): Una crisis financiera que filtró a las empresas de internet, dejando solo a las más sólidas y funcionales.

7. Web 2.0 y redes sociales (2004): Con el lanzamiento de Facebook, internet dejó de ser un lugar de solo lectura para convertirse en una plataforma participativa.

8. Lanzamiento del iPhone (2007): Marcó el inicio de la era del internet móvil, permitiendo estar conectados las 24 horas desde cualquier lugar.

9. El auge de la Computación en la nube (2010): Servicios como iCloud, Drive y AWS permitieron almacenar y procesar datos sin depender del hardware local.

10. Pandemia y trabajo híbrido: El internet se volvió la infraestructura crítica para la economía global, impulsando el desarrollo masivo de herramientas de videoconferencia y colaboración.

Herramientas Sincronas

Son aquellas en las que la comunicación sucede en tiempo real. Los participantes deben estar conectados simultáneamente.

Ejemplos: Videoconferencias (Zoom, Google Meet), Herramientas de voz y VOIP (WhatsApp calls, Skype), Chat en vivo (Soporte técnico por chat, Messenger).

Herramientas Asíncronas

La comunicación se produce diferida en el tiempo. No se requiere que emisor y receptor coincidan.

Ejemplos: Correo electrónico, Foros de discusión, sistemas de almacenamiento en la nube.

Herramientas Duales (Híbridas)

Son plataformas que permiten ambos tipos de comunicación dependiendo de cómo se utilicen.

Ejemplos: WhatsApp, Telegram, Discord.

Reporte Tema #3

18 02 26

Historia del HTML

El HTML no nació como un lenguaje de programación, sino como un sistema para organizar y compartir información científica.

Origen (1989-1991): Todo comenzó en el CERN (Suiza).

Tim Berners-Lee propuso un sistema de "Hipertexto" para que los científicos pudieran saltar de un documento a otro mediante enlaces. En 1991, publicó la primera descripción de HTML, que apenas tenía 18 etiquetas.

La guerra de los navegadores (90s): A medida que la web se hacía popular, empresas como Netscape y Microsoft empezaron a inventar sus propias etiquetas para competir. Esto causó un caos, porque una página se veía bien en un navegador pero mal en otro.

El estándar (W3C): Para poner orden, en 1994 se fundó el W3C. Ellos se encargaron de estandarizar el lenguaje para que todos los navegadores hablaran el mismo idioma.

El salto a HTML5 (2004): Tras años de usar versiones rígidas, llega el HTML5. Esta versión cambió todo al permitir integrar video, audio y gráficos 2D/3D directamente en el código, sin necesidad de complementos externos.

Estructura básica.

```

<!DOCTYPE html>
<html>
  <head>
    <title>Mi tarea</title>
  </head>
  <body>
    <h1>Hola mundo</h1>
  </body>
</html>

```


20 etiquetas y su función.

`<a>`: Crea enlaces a otras páginas

`<img>`: Inserta imágenes.

`<div>`: Define una sección o contenedor genérico.

`<span>`: Se usa para aplicar estilo a un fragmento de texto

`<ul>`: Crea una lista con viñetas

`<ol>`: Crea una lista numerada.

`<li>`: Representa un elemento dentro de una lista.

`<table>`: Sirve para crear tablas de datos.

`<tr>`: Define una fila dentro de una tabla.

`<td>`: Define una celda de datos en una tabla.

`<br>`: Realiza un salto de línea simple.

`<hr>`: Dibuja una línea horizontal

`<b>`: Pone el texto en negrita.

`<i>`: Pone el texto en cursiva.

`<button>`: Crea un botón que se puede pulsar.

`<input>`: Crea un campo donde el usuario puede escribir.

`<form>`: Define un formulario para enviar datos.

`<header>`: Define la parte superior de una página o sección.

`<footer>`: Define el pie de página.

`<section>`: Define una sección lógica de contenido.

Omar Joaquín Sosa Moreno 1447940 ~~IAS~~

Reporte Tema #4

20 02 26

3 formas de insertar estilos en HTML

Estilos en línea (inline): Se escribe directamente dentro de la etiqueta usando el atributo style. Es útil para cambios rápidos y únicos.

Ejemplo:

```
<P style="color: blue;"> Texto azul </p>
```

Estilos internos (internal): Se colocan dentro de la etiqueta <style>, que a su vez va dentro del <head> del documento. Afecta a toda esa página.

Ejemplo:

```
<style> body {background: grey;} </style>
```

Estilos externos (External): Es la forma más profesional. Se crea un archivo separado (ejemplo: estilos.css) y se vincula al HTML con la etiqueta <link>. Permite que un solo archivo de diseño controle muchas páginas a la vez.

Hojas de estilo CSS

CSS es el lenguaje que define la apariencia de un documento HTML. Mientras que el HTML es el "esqueleto" (la estructura), el CSS es la "piel" (colores, fuentes, tamaños y posiciones).

Conceptos clave de CSS

Selector: Es el elemento al que quieres aplicar el estilo (h1, p, .clase)

Declaración: Se compone de una propiedad (como el color) y un valor (como red)

Cascada: Se llama así porque los estilos se aplican por orden de prioridad.

Historia de Javascript

Javascript fue creado en 1995 por Brendan Eich mientras trabajaba en Netscape. Lo curioso es que se desarrolló en solo 10 días.

Nombre original: Al principio se llamó Mocha, luego LiveScript, y finalmente JavaScript por una estrategia de marketing (para aprovechar la fama de Java).

Estandarización: En 1997 se convirtió en un estándar bajo el nombre de ECMAScript (nombre oficial de la lengua hoy en día).

Evolución: Pasó de ser un lenguaje para hacer trucos visuales (como relojes o nieve cayendo) a ser el motor de aplicaciones complejas como Facebook, Google Maps y Netflix.

Ejemplos:

Alerta: Esta línea hace que aparezca una ventana de aviso en el navegador.

```
<script>
  alert("Mensaje de alerta");
</script>
```

Formulario: Crea una caja para escribir

```
<input type="text" id="nom" placeholder="Tu nombre">
<button onclick="hola()">Saludar</button>
<script>
  function hola() { let x = document.getElementById("nom").value;
    alert("Hola " + x); }
</script>
```


Amar Joaquín Sosa Moreno 1997990 IAS

Reporte Tema #6

PHP

Hypertext Preprocessor es un lenguaje de programación de código abierto del lado del servidor, diseñado principalmente para el desarrollo web con contenido dinámico. Fue creado originalmente por Rasmus Lerdorf en 1994 como un conjunto de scripts para rastrear visitas en su página personal, evolucionando rápidamente hacia un lenguaje completo.

A diferencia de tecnologías que se ejecutan en el navegador del usuario, el código PHP se procesa directamente en el servidor Web. Este toma las instrucciones, genera el código HTML resultante y lo envía limpio al navegador del usuario.

Características principales:

- Integración nativa: Se incrusta directamente dentro del código HTML, lo que facilita la creación de plantillas y páginas dinámicas.
- Conexión a bases de datos: Es altamente eficiente para comunicarse con sistemas de gestión de bases de datos, especialmente MySQL.
- Soporte masivo: Es la base de los sistemas de gestión de contenidos (CMS) más populares del mundo, como WordPress, lo que significa que gran parte de la web actual funciona gracias a PHP.

Reporte Tema #7

XML

Extensible Markup Language es un metalinguaje desarrollado por el W3C a finales de los años 90. Su propósito principal es almacenar, estructurar y transportar datos de manera que sean legibles tanto por seres humanos como por computadoras y aplicaciones informáticas.

A menudo se le compara con HTML porque ambos utilizan etiquetas (<...>); sin embargo, tienen objetivos completamente diferentes. Mientras que HTML fue diseñado para mostrar datos y enfocarse en la apariencia visual, XML fue creado exclusivamente para describir los datos y enfocarse en su contenido.

Puntos Clave

- Etiquetas personalizadas: No tiene etiquetas predefinidas. El desarrollador define sus propias etiquetas según las necesidades del negocio.
- Independencia de plataforma: Permite que sistemas de software totalmente diferentes e incompatibles intercambien información de manera segura y estandarizada.
- Estructura estricta: Requiere un diseño jerárquico y limpio, donde todas las etiquetas deben cerrarse correctamente y respetar el caso sensible.

Reporte Tema #8

Java

Es un lenguaje de programación de alto nivel, robusto y orientado a objetos, lanzado oficialmente al mercado por Sun Microsystems en 1995 bajo la dirección de James Gosling. Su diseño se basó en gran medida en la sintaxis de C y C++, pero eliminando elementos complejos que solían causar errores de memoria. La filosofía fundamental de Java se resume en la famosa frase: "Write once, run anywhere". Esto se logra gracias a la máquina virtual de Java. En lugar de compilar el código directamente para un sistema específico, Java lo convierte en un código intermedio llamado bytecode, el cual corre en cualquier dispositivo con JVM.

Áreas de aplicación:

- Aplicaciones empresariales: es el estándar para sistemas bancarios y plataformas corporativas de gran escala gracias a su alta seguridad y estabilidad.
- Desarrollo Móvil: Fue la base nativa del desarrollo de aplicación para el sistema operativo Android.
- Arquitectura de software: Su estricto enfoque en la programación orientada a objetos fomenta un código organizado, modular y altamente reutilizable.

Ornar Joaquín Sosa Moreno 1997840

Reporte Tema #9

.NET

Es una plataforma de desarrollo web gratuita, de código abierto y multiplataforma creada por Microsoft. Lanzada originalmente a principios de los años 2000 como una solución exclusiva para sistemas operativos Windows, evolucionó radicalmente para transformarse en un ecosistema moderno capaz de ejecutarse en Linux, macOS, iOS y Android. Esta plataforma proporciona un entorno unificado con herramientas de desarrollo, librerías robustas y soporte para múltiples lenguajes de programación de alto rendimiento, siendo C# el más representativo de ellos.

Componentes y ventajas

Versatilidad: Permite crear prácticamente cualquier tipo de software: aplicaciones web, desarrollo de videojuegos, apps móviles, etc.

Common Language Runtime: Es el motor encargado de gestionar la ejecución de los programas, manejando tareas críticas de forma automática como la asignación de memoria y la seguridad.

Ecosistema moderno: Su enfoque actual se centra en el rendimiento optimizado, la escalabilidad y una gran comunidad de código abierto que lo mantiene a la vanguardia tecnológica.

Amar Joaquín Sesa Norero 1997-10 IAS

Reporte General

Este reporte analiza cómo los temas anteriores se entrelazan de formas sistemáticas para constituir la infraestructura digital, el desarrollo de software y la navegación web contemporánea.

Conectividad e infraestructura Base

Todo el software moderno depende de la comunicación. La historia del internet y los modelos OSI y TCP/IP definen las reglas del juego: establecen el transporte físico y lógico de los datos, permitiendo que computadoras de todo el mundo se conecten y hablen el mismo idioma a nivel de red.

El ecosistema del Frontend

Una vez establecida la conexión, la interfaz de usuario cobra vida en el navegador web. La historia de HTML nos muestra cómo pasamos de documentos de texto plano a estructuras complejas. Este desarrollo se complementa con las 3 formas de insertar estilos para la máquina estética, y la historia de Javascript, que transforma páginas web estáticas en aplicaciones interactivas y dinámicas en tiempo real.

El ecosistema del Backend y datos

Detrás de lo que el usuario ve, existe

una máquina lógica y de almacenamiento. PHP actúa como el motor clásico y director del lado del servidor web. Para el manejo, intercambio y transporte de configuraciones o datos estructurados de forma universal, se utiliza XML.

Por otro lado, cuando las aplicaciones requieren una escala masiva, robustez corporativa o arquitecturas complejas de software de escritorio, móviles o empresariales, entran en juego plataformas y lenguajes de gran envergadura como Java y el ecosistema .NET.

Conclusión

El desarrollo de software actual no depende de una sola herramienta, sino de la integración limpia de estos componentes: una red sólida (TCP), una interfaz intuitiva, un backend eficiente y un formato de comunicación estandarizado.